Not approved alternative concept. This document is not the canonical Lux architecture, is not an approved plan, and is not under active development. Do not use it to guide implementation. See `docs/architecture/overview.md` for the current design.

Concept: Extension System Architecture for Lux

Exploring Registry-Based Element Dispatch and Self-Extension

April 2026 — Concept Stage

Contents

1 Overview

This document explores how Lux could evolve from a fixed element vocabulary to a self-extending display server. It sketches the extension system, service API, and in-display agent SDK that would make this possible. This is concept-stage work only. It is not an approved plan and is not under active development.

See also: `concept-self-extending-display.md` for the high-level vision, and `concept-prfaq.tex` for the product thinking.

1.1 Design Principles

P1. **Everything is an extension.** The core provides transport, dispatch, registry, frame management, render loop, and safety wrapper. Every user-visible element kind, applet, theme, and service is an extension.

P2. **Closed core, open extension surface.** The core is small and stable. Extensions are the growth vector. Open-Closed Principle applied at the system level.

P3. **Python is the substrate.** Not web. Extensions use Python's package ecosystem directly: matplotlib, plotly, PIL, pyte, Qt, tkinter. No transpilation to JavaScript.

P4. **Ollama-shaped service.** The primary interface is HTTP endpoints on localhost. Any agent, any language, any machine (with TCP transport) can drive the display. MCP is an adapter layer.

P5. **Self-extending by default.** When an agent needs a visualization that does not exist, it can author one. The extension persists for the user and optionally for the community.

P6. **Bidirectional.** The display is not passive. Agents write to Lux; Lux writes back (events, menu clicks, interaction messages, screenshots). The display can drive agent behavior.

1.2 Relationship to Current Lux

Lux currently has a fixed element vocabulary with a hardcoded ImGui dispatcher. This concept preserves the wire protocol shape, the frame system, the ImGui render loop, and the MCP adapter — but refactors element dispatch from hardcoded to registry-based and adds the extension system, service API, and in-display agent.

2 System Architecture

2.1 Component Map

Component	Responsibility
Core	
Transport	Unix socket (local) + TCP/TLS (remote). Length-prefixed JSON framing (inherited from current Lux).
Protocol	Message types, element schemas, serialization. Extended with extension-declared element kinds.
Registry	Runtime dispatch table: element kind → renderer. Populated by extension loader at startup, extensible at runtime.
Loader	Scans extension directories, validates manifests, imports renderers, registers with the registry. Crash-isolates each extension.
Frame Manager	Frame lifecycle, shared ownership, orphan model, dock bar, focus coordination. Inherited from current Lux.
Render Loop	ImGui event loop via `immapp.run()`. Polls sockets, dispatches to registry, flushes events. Single-threaded.
Safety Wrapper	Exception isolation per extension render call. Crash → error placeholder. Consent dialog on install.
Service Layer	
HTTP Server	Localhost HTTP endpoints (ollama-shaped). Runs in a thread; posts to main-thread queue.
/help	Auto-documenting endpoint. Returns installed element kinds, services, conventions, permission mode.
MCP Adapter	Thin translation: MCP tool calls → service API calls. Preserves Claude Code integration.
Extensions	
Starter Pack	Bundled extensions: text, button, table, plot, markdown, image, draw, etc. Ship with install.
User Local	`~/.lux/extensions/`. Persists across sessions.
Project Local	`.lux/extensions/`. Scoped to repo.
Community	Registry of published extensions. Installed via `lux ext install`.
Agent SDK (optional)	
LuxExchange	Tool-use loop (Anthropic Python SDK). Drives the in-display agent.
Commands	Leaf + composite tool commands operating on the display. Transactional composites with rollback.
Permission	Five-mode policy: Plan, Default, Accept-edits, Bypass, DontAsk.

2.2 Data Flow

```
External Agent (Claude Code, CLI, remote)
  |
  |  HTTP or MCP
  v
Service Layer (thread)
  |
  |  Queue (thread-safe)
  v
Main Thread: Render Loop
  |
  +-- Registry lookup: element kind -> extension renderer
  |
  +-- Extension renderer: imgui | texture | window
  |
  +-- Frame Manager: position, focus, dock
  |
  +-- Event flush: InteractionMessages -> all clients
```

3 Extension System

3.1 Extension Manifest

Every extension declares itself in a manifest file (`extension.py` or `extension.toml`) at the extension root:

```
# ~/.lux/extensions/sankey/extension.py

LUX_EXTENSION = {
    "name": "sankey",
    "version": "1.0.0",
    "author": "maya-torres",
    "description": "Sankey␣flow␣diagram",
    "element_kind": "sankey",
    "render_mode": "texture", # imgui | texture | window
    "schema": {
        "nodes": {"type": "list", "required": True},
        "edges": {"type": "list", "required": True},
        "title": {"type": "str", "required": False},
    },
    "dependencies": ["plotly>=5.0"],
    "lux_version": ">=2.0",
}

def render(element, ctx):
    """Render␣a␣sankey␣diagram."""
    import plotly.graph_objects as go
    fig = go.Figure(go.Sankey(
        node=dict(label=[n["label"] for n in element["nodes"]]),
        link=dict(
            source=[e["source"] for e in element["edges"]],
            target=[e["target"] for e in element["edges"]],
            value=[e["value"] for e in element["edges"]],
        ),
    ))
    fig.update_layout(title=element.get("title", ""))
    ctx.render_figure(fig) # texture mode: fig -> PNG -> texture
```

3.2 Three Rendering Modes

Each extension declares one of three rendering modes. The mode determines the `RenderContext` capabilities:

Mode	ctx Provides	When to Use
`imgui`	`imgui` module, `draw_list`, `widget_state`, `send_event()`	Native ImGui widgets, custom drawing, interactive controls. Preferred mode — single-window UX, full interactivity.
`texture`	`render_image(pil_img)`, `render_figure(mpl_fig)`, `render_bytes(png)`	Libraries that render to bitmaps: matplotlib, PIL, plotly (static export), any numpy→pixels pipeline. Displayed as ImGui image. Limited interactivity (click forwarding, no native zoom/pan).
`window`	`create_window(title)`, `on_close(callback)`, `send_event()`	Libraries that require their own OS window: Qt, tkinter, Jupyter kernels, Pharo (via Postern). Lifecycle tracked and focus coordinated but pixels not composited. Escape hatch for anything ImGui cannot host.

3.3 Extension Lifecycle

1. **Discovery.** Loader scans directories in order: bundled starter pack, project-local (`.lux/extensions/`), user-local (`~/.lux/extensions/`).

2. **Validation.** Manifest parsed. Schema checked. Dependencies verified via `importlib.metadata`. Incompatible `lux_version` → skipped with warning.

3. **Registration.** Element kind added to registry. Renderer function stored in dispatch table.

4. **Rendering.** When a scene contains an element with `kind` matching this extension, the registry dispatches to the extension's `render()` function with a mode-appropriate `RenderContext`.

5. **Crash isolation.** Every `render()` call is wrapped in `try/except`. Exception → error placeholder rendered in place; core keeps running. Repeated crashes → extension disabled for the session.

6. **Hot install.** `lux ext install` writes to user-local directory and signals the display process to re-scan. New extension available next frame. No restart required.

3.4 Extension Registry API

The registry is a Python dict at runtime:

```
_registry: dict[str, ExtensionEntry] = {}

@dataclass
class ExtensionEntry:
    name: str
    kind: str # element kind
    render_mode: str # "imgui" | "texture" | "window"
    schema: dict # field validation
    renderer: Callable # render(element, ctx)
    metadata: dict # author, version, description
    crash_count: int = 0 # for auto-disable
```

4 Service API

The concept proposes HTTP on localhost (default port 8423) as the primary interface. The shape follows Ollama: simple REST endpoints, JSON payloads, streaming where appropriate.

4.1 Endpoints

Method	Path	Purpose
POST	`/api/show`	Render a scene (elements, frame targeting).
POST	`/api/update`	Patch elements by ID.
GET	`/api/events`	Stream interaction events (SSE).
GET	`/api/frames`	Introspect current frame state.
POST	`/api/clear`	Clear all scenes and frames.
GET	`/api/ping`	Health check with round-trip time.
GET	`/api/extensions`	List installed extensions.
POST	`/api/extensions/install`	Install from path or URL.
POST	`/api/extensions/author`	LLM-authored extension (requires in-display agent).
DELETE	`/api/extensions/{name}`	Remove an extension.
GET	`/api/capabilities`	Installed kinds, services, render modes.
GET	`/help`	Auto-documenting: conventions, schemas, permission mode. Same contract as Postern's `/help`.
POST	`/api/screenshot`	Capture display as PNG. Returns base64 or writes to path.
POST	`/api/agent/run`	Submit a prompt to the in-display agent (requires Agent SDK).

4.2 The `/help` Contract

`/help` returns runtime truth from the live registry, not static documentation. An agent that has never seen this instance reads `/help` first and learns:

- What element kinds are installed right now
- What schemas each kind expects
- What rendering modes are active
- What permission mode is configured
- What extensions are available but not loaded (disabled, missing deps)
- Conventions and safety rules

This is the same pattern as Postern's `/help` endpoint: the live system self-documents.

4.3 MCP Adapter

The MCP server becomes a thin translation layer:

MCP Tool	Service Call
`show()`	`POST /api/show`
`update()`	`POST /api/update`
`recv()`	`GET /api/events` (single event)
`clear()`	`POST /api/clear`
`ping()`	`GET /api/ping`
`display_mode()`	`GET/POST /api/config/display`
`show_table()`	`POST /api/show` (convenience)
`show_dashboard()`	`POST /api/show` (convenience)
`show_diagram()`	`POST /api/show` (convenience)

Existing Claude Code integration is preserved. Agents that want the richer surface (extension management, streaming events, agent interaction) use the service API directly.

5 In-Display Agent SDK

Optional component. Same architecture as the Claude Agent SDK for Smalltalk: a tool-use loop where tools operate on the live display process.

5.1 LuxExchange

```
exchange = LuxExchange(
    client=anthropic.Client(),
    commands=[
        ShowSceneCommand(),
        InstallExtensionCommand(),
        AuthorExtensionCommand(),
        IntrospectFramesCommand(),
        ScreenshotCommand(),
        RunPythonCommand(), # consent-gated
    ],
    permission_policy=DefaultPolicy(),
    max_iterations=20,
)
exchange.run("Render this data as a sankey diagram")
```

5.2 Tool Commands

Following the Gang-of-Four Composite pattern from the Smalltalk SDK:

Command	Type	Mode	Purpose
ShowScene	Leaf	Safe	Render a scene.
UpdateScene	Leaf	Safe	Patch elements.
ClearScene	Leaf	Safe	Clear frames.
IntrospectFrames	Leaf	Safe	Dump current state.
ListExtensions	Leaf	Safe	Registry query.
ListCapabilities	Leaf	Safe	What kinds are available.
Screenshot	Leaf	Safe	Capture display as image.
InstallExtension	Leaf	Gated	Add extension (consent).
RemoveExtension	Leaf	Gated	Remove extension.
AuthorExtension	Leaf	Gated	Generate via LLM.
RunPython	Leaf	Gated	Exec in display namespace.
AuthorAndInstall	Composite	Gated	Author + install + verify. Rollback on failure.
RenderWithExt	Composite	Gated	Install if needed + render. Rollback on render failure.

5.3 Permission Policy

Five modes, same as the Smalltalk SDK:

Mode	Behavior
Plan	Show plan, do not execute. For review.
Default	Ask per gated call. Consent dialog.
Accept-edits	Auto-approve install/author. Ask for RunPython.
Bypass	Approve everything. For trusted pipelines.
DontAsk	Deny all gated calls. Safe mode.

5.4 Screenshot-Driven Iteration

The in-display agent can take a screenshot of its own output and feed it back to the LLM for visual QA:

1. Agent authors an extension and renders content.
2. Agent calls `ScreenshotCommand`.
3. Screenshot sent to Claude as an image content block.
4. Claude evaluates: "the columns are misaligned."
5. Agent modifies the extension and re-renders.
6. Repeat until visually correct.

This technique was proven at scale during the Postern/Pharo work, where agents iteratively refined Morphic UIs using screenshots of the running image.

6 Community Registry

6.1 Extension Distribution

Extensions are distributed as directories containing:

- `extension.py` or `extension.toml` — manifest
- `renderer.py` — render function (or inline in manifest)
- `requirements.txt` — Python dependencies
- `tests/` — test suite
- `README.md` — usage documentation

6.2 Registry Structure

A GitHub repository with a `registry.json` index:

```
{
  "extensions": [
    {
      "name": "sankey",
```

```
    "version": "1.0.0",
    "author": "maya-torres",
    "repo": "github.com/maya-torres/lux-ext-sankey",
    "ref": "v1.0.0",
    "render_mode": "texture",
    "rating": 4.2,
    "downloads": 347,
    "verified": {
      "lint": true,
      "ci": true,
      "tests": 12,
      "coverage": 0.85
    }
  }
 ]
}
```

6.3 CLI

Command	Purpose
`lux ext list`	List installed extensions.
`lux ext search <q>`	Search community registry.
`lux ext install <name>`	Install from registry.
`lux ext remove <name>`	Remove.
`lux ext publish`	Publish to registry (requires account).
`lux ext info <name>`	Show metadata, ratings, verification.

6.4 Verification Badges

Each community extension displays verification status:

- **Lint passing** — `ruff check` clean
- **CI green** — tests pass on latest Lux version
- **Test coverage** — percentage reported
- **Community rating** — 1–5 stars from users
- **Download count** — adoption signal
- **Author verified** — GitHub identity confirmed

The producer/consumer asymmetry is key: a small number of authors produce extensions consumed by a larger number of users who never need to author code themselves.

7 Security Model

7.1 Trust Boundaries

Boundary	Threat	Mitigation
Socket/HTTP	Unauthorized connection	Localhost binding by default. Token auth for TCP/TLS.
Extension install	Malicious code	Consent dialog shows source. Community verification badges.
Extension render	Crash or hang	Exception isolation. Auto-disable after repeated crashes.
RunPython command	Arbitrary exec	Consent-gated per permission mode. Same trust as Bash tool.
Community registry	Supply chain	Verification badges (lint, CI, tests). Pinned refs (git SHA).

7.2 Consent Model

Extension installation follows the same consent pattern as current Lux's render functions:

1. User or agent requests installation.
2. Display shows the extension source code in a consent dialog.
3. User clicks Allow or Deny.
4. Allowed: extension installs and registers.
5. Denied: extension is not loaded. No partial state.

Community extensions with high ratings and green verification badges may be auto-approved in Accept-edits or Bypass permission modes.

8 Migration Path

If this concept is pursued, a phased migration preserves backward compatibility:

1. **Phase 1: Registry.** Refactor element dispatch from hardcoded chain to registry lookup. All existing element kinds become built-in extensions loaded at startup. Wire protocol unchanged. MCP tools unchanged. Zero user-facing change.

2. **Phase 2: Service API.** Add HTTP server alongside existing Unix socket. MCP adapter calls service API internally. Both interfaces active.

3. **Phase 3: User extensions.** Enable `~/.lux/extensions/` loading. Ship reference extensions. `lux ext` CLI.

4. **Phase 4: Agent SDK.** Add LuxExchange, commands, permission policy. Optional — no impact on users who do not enable it.

5. **Phase 5: Community.** Launch registry. Publish starter pack + reference extensions. Verification pipeline.

Each phase is independently shippable. Phase 1 alone improves the codebase (removes hardcoded dispatch) without changing the product. Phase 3 is where the self-extending concept becomes a different product from current Lux.